

Lab 13 — LINQ Part 2 + EF Core (Code First)

Name: Saif Elden Khaled Nazmy Lotfy

Assignment: Lab 13 | ITI Summer Training — Web Development Using .NET

Lab ID: 31

Fingerprinted values for Lab ID 31

Part	Formula	Result
C — GPA threshold	$2.5 + ((31 \bmod 4) \times 0.3) = 2.5 + (3 \times 0.3)$	3.4
D — Instructor experience	$(31 \bmod 5) + 3 = 1 + 3$	4
G — Added property	$31 \bmod 3 = 1$	CreditsCompleted

Part A — Setup

Steps: Confirmed today's StudentPortalConsole project builds and runs with no errors, printing the Session 12 warm-up chain (Nada Samir, Yara Adel, Kareem Fouad) before any TODO was touched. Model classes, seed data, and EF Core package references were already present.

```
press run, and it
n immediately - before a
ch TODO sits exactly
the numbers run
```

```
C:\Users\PandaEyes\Desktop\C#\Lab13\bin\Debug\net10.0\StudentPortalConsole.exe
===== WARM-UP: Session 12's chain =====
Nada Samir
Yara Adel
Kareem Fouad
```

Part B — Predict-the-Output Drills

B1. Code

```
List<Student> empty = new List<Student>();
Console.WriteLine(empty.Count());
Console.WriteLine(empty.Any());
Console.WriteLine(empty.Average(s => s.Gpa));
```

Prediction: Count() → 0, Any() → False, Average() → throws **InvalidOperationException**, "Sequence contains no elements."

Reason: Count() and Any() have a well-defined answer for zero elements (0 and false), so they're safe. Average() has no sensible value for an empty set — LINQ refuses rather than inventing a 0.

B2. Code

```
foreach (var g in students.GroupBy(s => s.YearOfStudy))
{
    Console.WriteLine($"{g.Key} ");
}
```

Prediction: Prints **2 3 1 4** — not sorted.

Reason: GroupBy emits buckets in the order each key is first encountered while walking the source list. Yara (year 2) is first in the list, so bucket 2 comes out first, then 3 (Omar), then 1 (Nada), then 4 (Kareem).

B3. Code

```
var q = students.Where(s => s.YearOfStudy >= 3);
students.Add(new Student { FullName = "Test Person", YearOfStudy = 3, Gpa = 3.0 });
Console.WriteLine(q.Count());
```

Prediction: Prints **3**.

Reason: Deferred execution — q is only a description of a query when declared, and doesn't run until Count() forces it. By then "Test Person" (year 3) has already been added, so the query sees Omar(3), Kareem(4), and Test Person(3) = 3 matches.

```
===== PART B =====
// B1 Count: 0
// B1 Any: False
ConB1 Average threw: InvalidOperationException - "Sequence contains no elements"
B2: 2 3 1 4
// B3: 3
```

Part C — Aggregates and Grouping (Fingerprinted — threshold = 3.4)

C1. Seven Aggregates

Aggregate	Value
Total count	4
Count above threshold (3.4)	2 (Yara 3.5, Nada 3.9)

Average GPA	3.35
Highest GPA	3.9
Lowest GPA	2.8
Any below 2.0	False
All at/above 2.0	True

C2. Empty-Collection Exception (deliberate)

Count() → 0, Any() → False. Average() with no guard throws:

```
System.InvalidOperationException
"Sequence contains no elements"
```

C3. Guarded Fix

```
if (collection.Any())
    Console.WriteLine($"Average: {collection.Average(s => s.Gpa):F2}");
else
    Console.WriteLine("No students in the collection - average is undefined.");
```

C4. Group by YearOfStudy

Groups come out as 2, 3, 1, 4 — not sorted, because GroupBy orders buckets by first-encountered key, never by key value.

C5. Group by a Computed Key (my threshold, 3.4)

Two buckets — "Saif's Cutoff Met" (Yara 3.5, Nada 3.9) and "Saif's Cutoff Missed" (Omar 2.8, Kareem 3.2). Neither name is a stored field — the key is whatever the lambda returns.

C6. Repeat C4, Sorted

Added .OrderBy(g => g.Key) onto the GroupBy chain — groups now come out in order 1, 2, 3, 4.

```
===== PART C (threshold = 3.4) =====
Total count: 4
Count above my threshold (3.4): 2
(Average GPA: 3.35
Highest GPA: 3.9
Lowest GPA: 2.8
Any below 2.0: False
All at/above 2.0: True
C2 Count: 0
C2 Any: False
C2 threw: System.InvalidOperationException - "Sequence contains no elements"
C3: No students in the collection - average is undefined.
```

```
C4 - GroupBy YearOfStudy:
Year 2: 1 student(s)
Yara Adel
Year 3: 1 student(s)
Omar Hesham
Year 1: 1 student(s)
Nada Samir
Year 4: 1 student(s)
Kareem Fouad
C5 - GroupBy my own computed threshold band:
Saif's Cutoff Met: 2 student(s)
Yara Adel (3.50)
Nada Samir (3.90)
Saif's Cutoff Missed: 2 student(s)
Omar Hesham (2.80)
Kareem Fouad (3.20)
```

```
C6 - GroupBy YearOfStudy, sorted:
Year 1: 1 student(s)
Nada Samir
Year 2: 1 student(s)
Yara Adel
Year 3: 1 student(s)
Omar Hesham
Year 4: 1 student(s)
Kareem Fouad
```

Part D — Join and Its Silent Failure (Fingerprinted — experience = 4)

D1 & D2. Method Syntax and Query Syntax

```
var teaching = instructors.Join(
    courses,
    i => i.AssignedCourseName,
    c => c.CourseName,
    (i, c) => $"{i.FullName} teaches {c.CourseName} ({c.Credits} credits)");

var teachingQuery = from i in instructors
                    join c in courses on i.AssignedCourseName equals c.CourseName
                    select $"{i.FullName} teaches {c.CourseName} ({c.Credits} credits)";
```

Output (both): Hamdy teaches Web Development Using .NET (4 credits) / Mona Khalil teaches Database Fundamentals (3 credits)

D3. Added Myself

```
new Instructor {
    FullName = "Saif Elden Khaled Nazmy Lotfy",
    YearsOfExperience = 4, // (31 mod 5) + 3
    AssignedCourseName = "Machine Learning" // does not exist in courses
};
```

D4. Re-run the Join

3 instructors in, 2 rows out. My row vanishes silently — no error, no blank line — because Join keeps only items matching on both sides (inner-join semantics). A join finding no match is normal, expected behaviour, not an error condition.

D5. Getting the Unmatched Row Included

Would need a LEFT join, built in LINQ from GroupJoin + DefaultIfEmpty().

```
===== PART D (experience = 4) =====
D1 - method syntax:
Hamdy teaches Web Development Using .NET (4 credits)
Mona Khalil teaches Database Fundamentals (3 credits)
D2 - query syntax:
Hamdy teaches Web Development Using .NET (4 credits)
Mona Khalil teaches Database Fundamentals (3 credits)
D4 - Instructors in: 3, rows out: 2
```

```
y = new List<Student>();
B1 Count: {b1Empty.Count()});
B1 Any: {b1Empty.Any()});

e($"B1 Average: {b1Empty.Average
^

```

Part E — Deferred Execution and Your Own Operator

E1. Deferred Execution Proof

```
var query = students.Where(s => s.Gpa > 3.0); // nothing runs yet
students.Add(new Student { FullName = "Layla Mostafa", YearOfStudy = 2, Gpa = 3.7 });
Console.WriteLine(query.Count());
```

Prediction: 4

Actual result: 4 — the query only executes when Count() demands a result, and by then Layla is already in the collection.

E2 & E3. Multiple-Enumeration Bug (deliberate)

```
var highAchievers = students.Where(s => s.Gpa > 3.0); // no ToList()
Console.WriteLine(highAchievers.Count()); // runs #1
foreach (var s in highAchievers) Console.WriteLine(s.FullName); // runs #2
double avg = highAchievers.Average(s => s.Gpa); // runs #3
```

Comment: the filtering work actually runs **3 separate times** — once per consumption.

E4. The Fix

```
var fixedList = students.Where(s => s.Gpa > 3.0).ToList(); // runs ONCE
Console.WriteLine(fixedList.Count()); // property, no parentheses
foreach (var s in fixedList) Console.WriteLine(s.FullName);
double avg = fixedList.Average(s => s.Gpa);
```

Comment: matters far more once this shape points at a real database (Part H) — each extra enumeration there is a full network round-trip, not just wasted CPU cycles.

E5. My Own Extension Method — MyTopStudents()

```
public static class StudentQueryExtensions
{
    public static IEnumerable<Student> MyTopStudents(this IEnumerable<Student> source)
        => source.Where(s => s.Gpa >= 3.4); // Part C threshold
}

List<string> topNames = students
    .MyTopStudents()
    .OrderBy(s => s.FullName)
    .Select(s => s.FullName)
    .ToList();
```

Output: Nada Samir, Yara Adel.

E6. Deferred or Immediate?

Deferred — its body just returns a Where(...) call, and Where is deferred, so my operator inherits that behaviour automatically.

```
=====
===== PART E =====
E1 count (prediction was 4, actual is): 4
E3 count: 3
Collection: Yara Adel
            Nada Samir
            Kareem Fouad
Invalid: Kareem Fouad
than inv E3 avg: 3.53
```

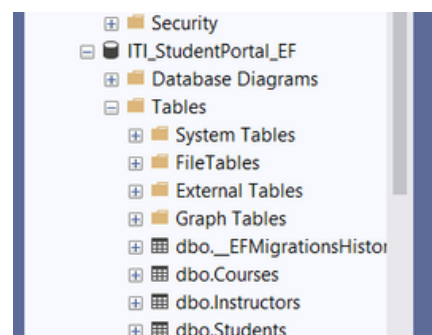
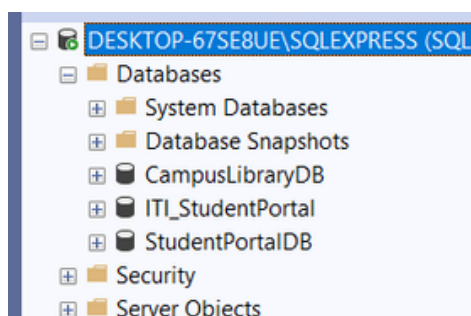
```
<Student>();
b1Empty.Count()");
Empty.Any()");
E4 count: 3
Yara Adel
Nada Samir
Kareem Fouad
E4 avg: 3.53
E5 - MyTopStudents() chain:
Nada Samir
Yara Adel
```

Part F — Your First Migration

- **Tables Up() creates:** Students, Courses, Instructors — one CreateTable per DbSet.
- **Column types:** Gpa (double) → SQL float. FullName (string) → SQL nvarchar(max). **Is FullName nullable?** No
- and i didnt write anything to make the EF do that
- **What Down() would do:** reverse Up() exactly — three DropTable calls (Instructors, Courses, Students).

Before Update-Database: SSMS confirms ITI_StudentPortalDB_EF does not exist yet — proving Add-Migration only writes a file and never touches the real database.

After Update-Database: database and tables appear in SSMS.



Two differences vs. the hand-built Session 3 table:

- 1. Column types — the hand-built table used explicit, sized types (e.g. varchar(50)); EF Code First defaults strings to nvarchar(max).
- 2. Naming/constraints — EF auto-generates its own PK constraint name (e.g. PK_Students) and identity column purely from the Id convention, versus an explicitly written, self-named PRIMARY KEY clause in Session 3.

Part G — Add Your Own Property and Migrate It (Fingerprinted — CreditsCompleted)

```
public class Student
{
    public int Id { get; set; }
    public string FullName { get; set; }
    public int YearOfStudy { get; set; }
    public double Gpa { get; set; }
    public int CreditsCompleted { get; set; }
    // 31 mod 3 = 1
}
```

Migration read before applying: Up() performs a single **AddColumn** — not a CreateTable. This matters because Students already holds real seeded rows: CreateTable would try to build a brand-new empty table (failing or wiping data), while AddColumn just appends a column and backfills the default (0) into every existing row.

After Update-Database: confirmed the new column exists, and row count is still **4** — the original seeded students survived.

ources

		Results	Messages			
		Id	FullName	YearOfStudy	Gpa	CreditsCompleted
1		1	Yara Adel	2	3.5	0
2		2	Omar Hesham	3	2.8	0
3		3	Nada Samir	1	3.9	0
4		4	Kareem Fouad	4	3.2	0

ability

er

Part H — Query the Database with LINQ

```
using (var context = new StudentPortalContext())
{
    if (!context.Students.Any())
    {
        context.AddRange(students);
        context.SaveChanges();
    }

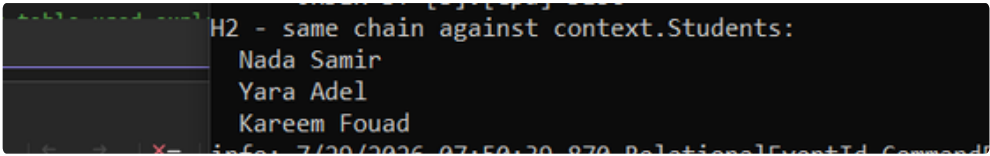
    var dbTopNames = context.Students
        .Where(s => s.Gpa > 3.0)
        .OrderByDescending(s => s.Gpa)
        .Select(s => s.FullName)
        .ToList();

    double dbAvg = context.Students.Average(s => s.Gpa);
    int dbCount = context.Students.Count();
}
```

Result: chain matches the in-memory Warm-Up character for character — Nada Samir, Yara Adel, Kareem Fouad. Average GPA = 3.35, count = 4.

Difference between the two lines:

- **context.Students.Where(...).ToList()** — filter first, so EF translates it into a SQL WHERE clause; filtering happens server-side. (This is the one used.)
- **context.Students.ToList().Where(...)** — ToList() first, pulling the entire table into memory before filtering client-side in C#. Same answer on 4 rows, drastically worse at scale.



Part I — Wrap-Up Reflection

- 1. **Lab ID 31**, three derived values: Part C threshold = 2.5 + ((31 mod 4) × 0.3) = 3.4; Part D experience = (31 mod 5) + 3 = 4; Part G property = 31 mod 3 = 1 → CreditsCompleted.
- 2. A silently missing join row is more dangerous than a crash because a crash announces itself and gets fixed the same day. A missing row looks exactly like a correct, complete answer — it can sit inside a report for months, quietly under-counting something that matters, with no error ever raised to prompt investigation.
- 3. Splitting Add-Migration and Update-Database into two commands is a safety feature: Add-Migration only ever writes a reviewable C# file, deletable with zero consequences via Remove-Migration. Nothing about the real database is at risk until Update-Database is deliberately run.
- 4. Running the identical LINQ chain against the database means context.Students is a DbSet<Student> — enumerating it sends a generated SQL query across the network rather than just looping in memory. Deferred execution matters more there because every extra enumeration costs a full network round-trip, and composing filters before consuming lets EF fold them into one efficient SQL statement instead of over-fetching data.